

Exercice 3 (6 points)

Cet exercice porte sur les structures de Files

Simon est un jeu de société électronique de forme circulaire comportant quatre grosses touches de couleurs différentes : rouge, vert, bleu et jaune. Le jeu joue une séquence de couleurs que le joueur doit mémoriser et répéter ensuite. S'il réussit, une couleur parmi les 4 est ajoutée à la fin de la séquence. La nouvelle séquence est jouée depuis le début et le jeu continue. Dès que le joueur se trompe, la séquence est vidée et réinitialisée avec une couleur et une nouvelle partie commence.



Source Wikipédia

Exemple de séquence jouée : rouge → bleu → rouge → jaune → bleu

Dans cet exercice nous essaierons de reproduire ce jeu.

Les quatre couleurs sont stockées dans un tuple nommé `couleurs` :

```
couleurs = ("bleu", "rouge", "jaune", "vert")
```

Pour stocker la séquence à afficher nous utiliserons une structure de file que l'on nommera `sequence` tout au long de l'exercice.

La file est une structure linéaire de type FIFO (First In First Out). Nous utiliserons durant cet exercice les fonctions suivantes :

Structure de données abstraite : File

<code>creer_file_vide()</code>	: renvoie une file vide
<code>est_vide(f)</code>	: renvoie <code>True</code> si <code>f</code> est vide, <code>False</code> sinon
<code>enfiler(f, element)</code>	: ajoute <code>element</code> en queue de <code>f</code>
<code>defiler(f)</code>	: retire l'élément en tête de <code>f</code> et le renvoie
<code>taille(f)</code>	: renvoie le nombre d'éléments de <code>f</code>

En fin de chaque séquence, le Simon tire au hasard une couleur parmi les 4 proposées. On utilisera la fonction `randint(a,b)` de la bibliothèque `random` qui permet d'obtenir un nombre entier compris entre `a` inclus et `b` inclus pour le tirage aléatoire. Exemple : `randint(1, 5)` peut renvoyer 1, 2, 3, 4 ou 5.

1.

Recopier et compléter, sur votre copie, les [...] des lignes 3 et 4 de la fonction `ajout(f)` qui permet de tirer au hasard une couleur et de l'ajouter à une séquence. La fonction `ajout` prend en paramètre la séquence `f` ; elle renvoie la séquence `f` modifiée (qui intègre la couleur ajoutée au format chaîne de caractères).

1	def <code>ajout(f)</code> :
2	<code>couleurs = ("bleu", "rouge", "jaune", "vert")</code>
3	<code>indice = randint([...],[...])</code>
4	<code>enfiler([...],[...])</code>
5	return <code>f</code>

En cas d'erreur du joueur durant sa réponse, la partie reprend au début ; il faut donc vider la file `sequence` pour recommencer à zéro en appelant `vider(sequence)` qui permet de rendre la file `sequence` vide sans la renvoyer.

2.

Ecrire la fonction `vider` qui prend en paramètre une séquence `f` et la vide sans la renvoyer.

Le Simon doit afficher successivement les différentes couleurs de la séquence. Ce rôle est confié à la fonction `affich_seq(sequence)`, qui prend en paramètre la file de couleurs `sequence`, définie par l'algorithme suivant :

- on ajoute une nouvelle couleur à `sequence` ;
- on affiche les couleurs de la séquence, une par une, avec une pause de 0,5 s entre chaque affichage.

Une fonction `affichage(couleur)` (dont la rédaction n'est pas demandée dans cet exercice) permettra l'affichage de la couleur souhaitée avec `couleur` de type chaîne de caractères correspondant à une des 4 couleurs.

La temporisation de 0,5 s sera effectuée avec la commande `time.sleep(0.5)`. Après l'exécution de la fonction `affich_seq`, la file `sequence` ne devra pas être vidée de ses éléments.

3.

Recopier et compléter, sur la copie, les [...] des lignes 4 à 10 de la fonction `affich_seq(sequence)` ci-dessous :

1	def <code>affich_seq(sequence)</code> :
2	<code>stock = creer_file_vide()</code>
3	<code>ajout(sequence)</code>
4	while <code>not est_vide(sequence)</code> :
5	<code>c = [...]</code>
6	<code>[...]</code>
7	<code>time.sleep(0.5)</code>
8	<code>[...]</code>
9	while <code>[...]</code> :
10	<code>[...]</code>

4.

Cette question est indépendante des précédentes : bien qu'elle fasse appel aux fonctions construites précédemment, elle peut être résolue même si le candidat n'a pas réussi toutes les questions précédentes.

Nous allons ici créer une fonction `tour_de_jeu(sequence)` qui gère le déroulement d'un tour quelconque de jeu côté joueur. La fonction `tour_de_jeu` prend en paramètre la file de couleurs `sequence`, qui contient un certain nombre de couleurs.

- Le jeu électronique Simon commence par ajouter une couleur à la séquence et affiche l'intégralité de la séquence.
- Le joueur doit reproduire la séquence dans le même ordre. Il choisit une couleur via la fonction `saisie_joueur()`.
- On vérifie si cette couleur est conforme à celle de la séquence.
- S'il s'agit de la bonne couleur, on poursuit sinon on vide `sequence`.
- Si le joueur arrive au bout de la séquence, il valide le tour de jeu et le jeu se poursuit avec un nouveau tour de jeu, sinon le joueur a perdu et le jeu s'arrête.

La fonction `tour_de_jeu` s'arrête donc si le joueur a trouvé toutes les bonnes couleurs de `sequence` dans l'ordre, ou bien dès que le joueur se trompe.

Après l'exécution de la fonction `tour_de_jeu`, la file `sequence` ne devra pas être vidée de ses éléments en cas de victoire.

- a. Afin d'obtenir la fonction `tour_de_jeu(sequence)` correspondant au comportement décrit ci-dessus, recopier le script ci-dessous et :

- Compléter le `...`
- Choisir parmi les propositions de syntaxes suivantes lesquelles correspondent aux ZONES A, B, C, D, E et F figurant dans le script et les y remplacer (il ne faut donc en choisir que six parmi les onze) :

```
vider(sequence)
defiler(sequence)
enfiler(sequence, c_joueur)
enfiler(stock, c_seq)
enfiler(sequence, defiler(stock))
enfiler(stock, defiler(sequence))
affich_seq(sequence)
while not est_vide(sequence):
while not est_vide(stock):
if not est_vide(sequence):
if not est_vide(stock):
```

```

1  def tour_de_jeu(sequence):
2      ZONE A
3      stock = creer_file_vide()
4      while not est_vide(sequence) :
5          c_joueur = saisie_joueur()
6          c_seq = ZONE B
7          if c_joueur [...] c_seq:
8              ZONE C
9          else:
10             ZONE D
11     ZONE E
12     ZONE F

```

- b. Proposer une modification pour que la fonction se répète si le joueur trouve toutes les couleurs de la séquence (dans ce cas, une nouvelle couleur est ajoutée) ou s'il se trompe (dans ce cas, la séquence est vidée et se voit ajouter une nouvelle couleur). On pourra ajouter des instructions qui ne sont pas proposées dans la question a.